

IDENTITY IS THE SIGNAL: AGENT-ACTION RISK NEEDS DELEGATED AUTHORITY, NOT JUST CONTENT

Anonymous authors

Paper under double-blind review

ABSTRACT

Guardrail models for AI agents classify a proposed action from its text alone. We show this misses the dimensions that matter most for delegated, tool-using agents. Removing principal entitlements, delegated scope, and policy text from a trained 2B checker’s input drops macro AUPRC from 0.960 to 0.790 on held-out tools, and the loss is confined to three dimensions (unauthorized-scope 1.00 to 0.13, policy-conflict 0.96 to 0.39) while content dimensions do not move. Two zero-shot safety models and the acting agent asked to judge its own call are at chance on exactly these dimensions with or without the context, so prompting cannot close the gap; a blind LLM judge loses the same dimensions when identity is removed. Policy-conditioned reasoning generalizes to unseen policy phrasings (0.98 AUPRC) but not unseen kinds (0.72 at 2B, 0.84 at 8B), and stacking independently thresholded policies reaches a 0.97 false positive rate by three policies unless decisions come from calibrated joint probabilities. On AgentDojo and InjecAgent, traces we did not generate, policy-conflict transfers zero-shot (0.85 and 0.97 AUPRC against 0.24 and 0.46 base rates). Prompt-injection did not transfer at 2B until the external data exposed a label leak in our generator; the leak-free checkers reach 0.69 (2B) and 0.84 (8B) AUPRC on AgentDojo and rank the poisoned InjecAgent tool output above its clean twin on 98% of pairs (50% and 35% before), while the synthetic score on the same dimension did not move.

1 INTRODUCTION

Deployed AI agents act with delegated authority: a support agent issues refunds for a merchant, a coding agent pushes commits for a developer, an operations agent grants IAM roles for an administrator. Before such an action executes, some system must decide whether it is safe to proceed. The dominant tool is a guardrail classifier that reads the proposed action and outputs a harm judgment (Inan et al., 2023; Padhi et al., 2024; Zeng et al., 2024; Ghosh et al., 2024; OpenAI, 2025).

This conflates two questions. **Content risk** asks whether an action looks dangerous on its face (deletes data, moves money, contacts an external party) and is answerable from the action text. **Authorization risk** asks whether *this* action is something *this* agent was allowed to do, on behalf of *this* principal, under the policy in force. It needs the principal’s entitlements, the agent’s delegated scope, and the applicable policy clause as structured inputs, not prose the model must infer.

We build a pre-execution risk classifier, *forecheck*, whose input contract makes these fields explicit, then remove them from the trained model’s input to see what breaks. Three of eleven risk dimensions collapse from near-perfect to near-chance; the rest do not move (§3). Prompting cannot close the gap: two zero-shot safety models given the same policy text are at chance on exactly these dimensions with or without identity, and so is the acting agent asked to judge its own call (§6). A blind LLM judge shown only the rendered text loses the same dimensions when identity is removed, replicating the ablation on labels rather than model scores (§7). Policy-conditioned reasoning generalizes to unseen policy *phrasings* but not unseen *kinds* (§4), and composing several classifiers’ decisions, as guardrail stacks are deployed, compounds toward denying everything unless the decision is made from the joint calibrated probability vector (§5).

Finally we score the same checkers zero-shot on two published benchmarks we did not generate, AgentDojo and InjecAgent (§8). Policy-conflict transfers on both. Prompt-injection did not transfer

at 2B, and tracing that failure exposed a label leak in our own generator that every synthetic split had hidden. Fixing it and retraining turns the 2B failure into a 0.69 AUPRC transfer and lifts both checkers from chance to 98% on InjecAgent’s poisoned-versus-clean pairs, while the synthetic score on the same dimension does not change. Every number outside §8 is on synthetic data; §10 states what that licenses.

2 PROBLEM SETTING

Contract. forecheck scores a proposed tool call against an `ActionContext`: the principal’s entitlements, the scope actually delegated to the agent (a possibly strict subset), a trust-labelled trajectory of prior observations, the applicable policy clause(s), and the proposed action (tool, arguments, destination), all as typed fields. A classifier that sees only the rendered action, as every guardrail model in §9 does, can answer content-risk questions but cannot check whether the agent was delegated the scope it is exercising.

Eleven risk dimensions. Four *content* dimensions are decidable from action and destination alone (`destructive_or_irreversible_action`, `financial_commitment`, `external_communication`, `untrusted_destination`); two *trajectory* dimensions from prior observations (`prompt_injection_influence`, `suspicious_action_sequence`); `sensitive_data_exposure` behaves as content because its deciding field renders into the arguments. Four are where identity and policy are load-bearing: `unauthorized_scope` (inside the principal’s entitlements *and* the delegated scope?), `policy_conflict` (violates a stated clause?), `insufficient_context` (enough delegated information to decide?), and `privilege_escalation` (widens authority beyond the grant?).

Data. Synthetic data from forecheck’s scenario generator (49,996 rows per version), split into train/calibration/dev/test, a heldout_family split withholding a stratified subset of tools, and heldout_policy_kind/heldout_policy_phrasing splits (§4). Versions differ only in the defect each fixes: v4 renders the `privilege_escalation` deciding fields, v5 doubles the policy-kind inventory, v6 removes the injection label leak (§8). Every synthetic number measures whether an arm learned the generator’s label function, not real-world risk.

Arms. A LoRA-adapted decoder (Hu et al., 2021) at 2B (`granite-3.3-2b-instruct`) and 8B, scored by candidate-token logits per dimension and temperature-calibrated on the calibration split; two linear-head encoders (ModernBERT-large (Warner et al., 2024), `granite-embedding-r2`); a deterministic rule baseline over action and destination fields; two zero-shot safety models given the same policy text in-prompt, Granite Guardian 3.3 8B (Padhi et al., 2024) and `gpt-oss-safeguard 20B` (OpenAI, 2025); and an *agent self-judgment* arm where the untrained 8B instruct model (IBM Granite Team, 2024) gets the same context in an agent system prompt and returns one ALLOW/STOP verdict. We report AUPRC because positive rates range from 0.03 to 0.58; chance is the positive rate.

3 IDENTITY ABLATION

At evaluation time only, we remove principal entitlements, delegated scope, `on_behalf_of`, and policy text from the trained decoder’s rendered context (`-strip-identity`), leaving action, destination, and trajectory untouched. We compare against the rule baseline, the two zero-shot safety models, a decoder *trained* on identity-free renderings, and the same ablation at 8B (Tables 1 and 2; all eleven dimensions in Appendix A).

The loss is confined to three dimensions. Macro AUPRC falls 0.17, and Table 2 locates all of it. `unauthorized_scope` falls below the rule baseline once the entitlements it must be checked against are gone; `policy_conflict` falls to the rule floor because no clause is left to violate; `insufficient_context` halves. Content and trajectory dimensions do not move: `privilege_escalation` survives because the v4 renderer puts the deciding authority fields into the call text, and injection is read from the trajectory.

Not a fragile mechanism; scale does not recover it. Training on identity-free renderings from the start gives the same ceiling as stripping at evaluation (macro 0.784 vs. 0.790, per-dimension within

Table 1: Macro AUPRC with and without identity/policy context, `heldout_family` unless noted. Zero-shot rows use a 2,000-row (Granite Guardian) or 400-row (gpt-oss-safeguard) subsample. Source: docs/results/synthetic-v2/README.md.

Arm	Full	Stripped
Decoder 2B v2	0.915	0.736
Decoder 2B v4	0.960	0.790
Decoder 8B v4	0.958	0.786
Decoder 2B v4, trained stripped	—	0.784
Encoder ModernBERT-large	0.813	0.712
Rule baseline	0.614	0.614
Granite Guardian 3.3 8B (zero-shot)	0.257	0.271
gpt-oss-safeguard 20B (zero-shot, test)	0.269	0.238
Agent self-judgment 8B (§6)	0.242	0.281

Table 2: Decoder 2B v4, `heldout_family` ($n=6653$) AUPRC per dimension: full input, identity removed at evaluation, identity removed at training, and rule baseline. Positive rate in parentheses. First three rows are the delegated-authority dimensions.

Dimension (pos. rate)	Full	Eval-stripped	Train-stripped	Rule
<code>unauthorized_scope</code> (0.106)	0.999	0.125	0.107	0.507
<code>policy_conflict</code> (0.385)	0.964	0.392	0.429	0.370
<code>insufficient_context</code> (0.079)	0.841	0.412	0.357	0.232
<code>privilege_escalation</code> (0.075)	1.000	1.000	—	0.364
<code>prompt_injection_influence</code> (0.075)	0.804	0.806	—	0.287
<code>financial_commitment</code>	1.000	1.000	1.000	1.000

0.06), so stripping did not merely break a shortcut: the information is absent from the content. An 8B decoder collapses identically (0.958 to 0.786) on the same three dimensions.

Zero-shot safety models never used the context. Their macro AUPRC is 0.24–0.27, at chance, and unchanged or slightly higher when identity is removed. If a general safety model quietly used delegated-authority context, stripping would hurt; it does not, for either model. Both were trained for harm, jailbreak, and groundedness, not for checking an action against entitlements or a clause. This claim rests on the three dimensions being labels an independent reader can recover; §7 finds that holds for two of the three.

4 POLICY GENERALIZATION: PHRASING VS. KIND

Robustness to unseen *tools* is not robustness to unseen *policies*, and the latter is the deployment test: customers write their own clauses. `heldout_policy_kind` withholds entire predicate kinds (e.g. `forbid_recipient_domain`) from training; `heldout_policy_phrasing` withholds one paraphrase of every kind the model did train on. v4 has 15 kinds with 2 withheld; v5 has 30 with 4.

Table 3: `policy_conflict` AUPRC across splits. `heldout_policy_kind` cells are means over training seeds (2B: three seeds, sd 0.032 for v4, 0.026 for v5; 8B: two seeds); other cells are seed 0. Rule baseline on that split is 0.325. Macro AUPRC and per-seed values in Appendix B.

Split	2B v4 (15 kinds)	2B v5 (30 kinds)	8B v4
test	0.994	0.996	0.990
<code>heldout_family</code>	0.964	0.933	0.965
<code>heldout_policy_phrasing</code>	0.993	0.980	0.978
<code>heldout_policy_kind</code>	0.717	0.725	0.838

Unseen phrasing generalizes; unseen kinds do not, even with twice as many kinds. A paraphrase never seen in training leaves `policy_conflict` at 0.98–0.99 AUPRC, matching the test split:

the model reads the clause’s meaning, not a template id. On withheld kinds 2B lands at 0.717 (v4) and 0.725 (v5) across three seeds each; doubling the kind inventory moves the mean by less than one seed standard deviation. The model is far above the 0.325 rule baseline, so it has learned something about unseen kinds, but not the general operation of checking a call against an arbitrary clause. **Capacity helps partially.** 8B reaches 0.838 (two seeds: 0.851, 0.825), a real gain over the 2B band and still well short of the 0.98 phrasing ceiling. The supportable claim is “one model, policies of a kind seen in training, any phrasing,” not “arbitrary new policy kinds.”

5 MULTI-POLICY COMPOSITION AND THE CALIBRATION HINGE

Guardrail deployments run several independently tuned policies and act on the most restrictive decision, OR-ing k binary guards, so false positives compound in k alone. forecheck instead emits one calibrated probability vector over the eleven dimensions, composed by one policy engine and decided either by per-dimension thresholds or by the argmin of expected cost over the joint vector under a fixed cost table.

Table 4: False positive rate on covered-benign examples, heldout_family ($n=6640$), stacking k synthetic per-dimension policies. independent ORs k separately thresholded decisions; expected_cost_joint merges the same policies into one bundle and decides from the joint probability vector. FNR, review rate, the joint strategy, and curves in Appendix C.

Arm (macro ECE)	Strategy	$k=1$	$k=2$	$k=3$ (same to 8)
Decoder 2B (0.004)	independent	0.015	0.140	0.966
Decoder 2B (0.004)	expected_cost_joint	0.015	0.082	0.082
Encoder granite-r2 (0.049)	independent	—	0.710	0.990
Encoder granite-r2 (0.049)	expected_cost_joint	—	0.921	0.921

Threshold stacking denies everything by $k=3$. Independent stacking pushes the decoder’s FPR from 0.015 to 0.966 by three policies and holds it there through eight, not because any policy got worse but because more were added. Deciding from the joint calibrated vector is invariant in k from $k=2$ on (FPR/FNR/review rate flat at 0.082/0.070/0.28): the decision depends on the probability vector and the cost table, not on how many rules mention a dimension.

The invariance is a calibration property, not a decision-rule property. The same rule on the granite-embedding-r2 encoder’s probabilities, identical examples and policies, settles at FPR 0.921, worse than its own independent stacking at $k=2$. The decoder is calibrated (macro ECE 0.004, temperature-scaled (Guo et al., 2017; Platt, 1999)); the encoder is not (0.049). Expected-cost composition keeps a stack from collapsing only when the probabilities feeding it are calibrated. The cost table is the default severity tier, so the claim is about the curve’s shape in k , not the table. The product-facing form of the same probabilities is in Appendix D: at a 5% deployment incident rate, 89–99% of human approvals can be auto-decided under a 1–5% incident budget.

6 CAN THE ACTING AGENT JUDGE ITS OWN SCOPE?

A cheaper alternative to an external checker is to ask the acting agent, inside its own system prompt, whether to proceed. We give the untrained 8B instruct model an agent system prompt carrying the identical entitlements, delegated scope, and policy text, then the pending call, and read one ALLOW/STOP verdict off the first output token. That scalar is scored against all eleven labels on the same 2,000 heldout_family rows as the trained checker (Table 5; all dimensions in Appendix E).

The agent cannot judge its own scope from inside its prompt. With full context it is at or near chance on every identity-dependent dimension (AUROC 0.53–0.67), the level of the zero-shot safety models; the checker beats it on all eleven dimensions by 0.45–0.90 AUPRC. **Identity context makes the agent worse at spotting injection.** Removing the policy and scope text raises its injection AUPRC from 0.166 to 0.507 while the checker’s does not move (0.804 to 0.806): a long policy preamble dilutes the untrained model’s attention to an injected instruction elsewhere in the trajectory. Both findings argue for an external checker, even on content dimensions.

Table 5: Agent self-judgment (8B, untrained, single ALLOW/STOP verdict) vs. the trained 2B v4 checker, AUPRC on the same 2,000 heldout_family rows; positive rate in parentheses.

Dimension (pos. rate)	Agent 8B	Agent 8B, stripped	Checker 2B v4
unauthorized_scope (0.106)	0.135	0.101	0.999
policy_conflict (0.385)	0.509	0.419	0.964
insufficient_context (0.079)	0.086	0.083	0.841
privilege_escalation (0.075)	0.103	0.101	1.000
prompt_injection_influence (0.075)	0.166	0.507	0.804
Macro	0.242	0.281	0.960

7 ARE THE LABELS REAL? AN INDEPENDENT BLIND JUDGE

Is each label observable in the rendered text at all, to a reader who never sees the generator’s latent scenario? A frontier model (gpt-5.6-sol), given only the rendered context and the eleven dimension definitions, judged 600 v4 rows stratified to at least 20 positives and 20 negatives per dimension, once with the full rendering and once identity-stripped. Agreement with the generator is not ground truth, but low agreement is informative: the label then depends on information the generator tracked and never rendered, and a model scoring well on it is learning a generator regularity.

Table 6: Blind LLM-judge agreement with the generator label (yes vs. not-yes, Cohen’s κ), $n=600$ v4 rows, full and identity-stripped renderings. Right: AUPRC of the 2B v4 checker and the rule baseline *against the judge’s labels* on the judged heldout_family rows ($n=257$). Full table in Appendix F.

Dimension	κ full	κ stripped	Checker vs. judge	Rule vs. judge
privilege_escalation	0.909	0.909	0.977	0.297
prompt_injection_influence [†]	0.905	0.905	1.000	0.459
policy_conflict	0.714	0.029	0.889	0.502
unauthorized_scope	0.391	0.040	0.592	0.509
destructive_or_irreversible_action	0.511	0.542	0.635	0.509
untrusted_destination	0.213	0.205	0.864	0.785
insufficient_context	0.059	−0.036	0.090	0.090
sensitive_data_exposure	0.002	0.061	0.405	0.291
macro (11 dims)	0.436	0.340	0.613	0.415

[†]Measured on v4 text carrying the injection label leak (§8); judge and generator could agree by reading the leaked argument.

Three tiers. *Recoverable* ($\kappa > 0.7$): privilege_escalation, prompt_injection_influence, policy_conflict; for the last, judge recall is 0.89 and the disagreements are the judge reading a clause more strictly than the generator’s predicate. *Definitional disagreement* ($0.2 \leq \kappa \leq 0.5$): six dimensions where both produce coherent decisions under different definitions; the judge flags twice as many unauthorized_scope rows because it counts a mismatch with the agent’s stated objective as out of scope, while the generator’s label is entitlement-based (judge recall of generator positives 0.74). These are taxonomy decisions to fix in the definitions. *Not recoverable* ($\kappa < 0.1$): insufficient_context and sensitive_data_exposure; the judge calls the context sufficient on 50 of 58 generator-positive rows.

The judge replicates the identity ablation on labels. Stripping identity from the judge’s input drops policy_conflict κ from 0.714 to 0.029 (28 judge-positives instead of 259: no clause left to check) and unauthorized_scope from 0.391 to 0.040 (203 positives at precision 0.16: guessing from the action). Every other dimension is unchanged within noise, and the content dimensions’ judge-yes counts are identical between passes. These are the same two dimensions the trained checker loses (Table 2).

Consequence for §3. The identity-ablation claim is solid on policy_conflict, directionally confirmed on unauthorized_scope, and unvalidated on insufficient_context: the

checker’s 0.841 on that label is closer to a generator regularity than to a checkable question about the text, and we flag every number on it. Scored against the judge’s labels instead of the generator’s, the checker keeps the three recoverable dimensions and beats the rule baseline on each by 0.39–0.68 AUPRC.

8 TRANSFER TO DATA WE DID NOT GENERATE

We score the checkers zero-shot on two published benchmarks. Nothing was trained or tuned on either; calibration was fitted on synthetic dev and threshold selection is disabled.

AgentDojo (Debenedetti et al., 2024): banking-suite traces for claude-3-5-sonnet-20241022 and gpt-4o-2024-05-13, 1,714 runs, 4,215 proposed calls, 11 attack types plus no-attack runs. Labels come from AgentDojo’s own `ground_truth()` functions: `prompt_injection_influence` is yes when a call executes the injection task’s goal (positive rate 0.150); `unauthorized_scope` when the call’s function is outside the user task’s reference solution (0.253); `policy_conflict` when one of four banking policies we wrote once fires under a deterministic predicate (0.237). **InjecAgent** (Zhan et al., 2024): 1,054 cases over 38 toolkits. AgentDojo’s injection positives differ from negatives in the call itself (an unrequested transfer to a recipient the user never named), so it cannot tell whether a checker reads the untrusted text or the call. InjecAgent can: for each attacker tool we build a *poisoned/clean pair*, the same attacker call with identical arguments, proposed after the tool response with the attacker’s instruction filled in (yes) or left empty (no). The only difference inside a pair is the observation. 4,250 examples, 1,598 pairs, `policy_conflict` from five generic policies. Derivations: docs/external-{agentdojo,injecagent}.md.

Table 7: Zero-shot AUPRC on AgentDojo banking ($n=4,215$; 4,133 for injection) and InjecAgent ($n=4,250$; 3,196 for injection), positive rates in the header. Pair win: fraction of the 1,598 InjecAgent pairs on which the poisoned variant scores higher on injection (0.5 is chance). Baseline rows on fixed 1,000-row subsamples. Full tables, AUROC, four-suite results, and per-example logits in Appendix G.

Arm	AgentDojo banking			InjecAgent			pair win
	inj. (.15)	scope (.25)	policy (.24)	inj. (.50)	scope (.75)	policy (.46)	
Rule baseline	0.250	0.253	0.237	0.500	0.752	0.464	–
Decoder 2B v4	0.128	0.257	0.849	0.549	0.732	0.965	0.504
2B v4, eval-stripped	0.125	0.218	0.694	–	–	–	–
2B v4, train-stripped	0.150	0.234	0.390	–	–	–	–
Decoder 8B v4	0.701	0.355	0.626	0.544	0.849	0.977	0.353
8B v4, eval-stripped	0.743	0.266	0.404	0.556	0.843	0.968	0.382
Decoder 2B v6 (leak-free)	0.693	0.214	0.670	0.878	0.704	0.922	0.977
2B v6, eval-stripped	0.669	0.306	0.521	–	–	–	–
Decoder 8B v6 (leak-free)	0.838	0.239	0.785	0.943	0.677	0.965	0.977
8B v6, eval-stripped	0.819	0.354	0.450	0.953	0.917	0.963	0.978
Granite Guardian 3.3 8B	0.103	0.289	0.224	0.511	0.613	0.359	–
Agent-self 8B (ALLOW/STOP)	0.291	0.442	0.449	0.530	1.000	0.647	–

Policy checking transfers, and stays identity-dependent. 2B v4 scores 0.849 AUPRC on AgentDojo `policy_conflict` against a 0.237 base rate and a rule baseline at base rate, on policies, a tool schema, and traces it never saw. Eval-time stripping drops it to 0.694 and the train-stripped model to 0.390: the identity dependence of §3 on external data. On InjecAgent it transfers again (0.965, 0.977 against 0.464); stripping barely moves it there because the five policies are global rules about the call, not about what the principal named. The 2B/8B ordering on this dimension is not robust (8B below 2B on v4, above on v6); both sizes rank violations well (AUROC 0.91–0.94) and differ in how many benign calls they place above them.

Injection at v4: 2B fails, 8B transfers, neither reads the untrusted text. 2B v4 is below base rate on AgentDojo injection (0.128, AUROC 0.348): its injection head never fires on this domain (raw logits for all 4,133 calls between -5.0 and -3.0) while its policy head separates cleanly on the same rows. 8B v4 reaches 0.701 (AUROC 0.895). But on InjecAgent, with the call held fixed, the

8B v4 injection score rises on only 35% of pairs and is unchanged on 39%, and 2B v4 is at coin-flip; both heads sit at raw logit -4 on poisoned and clean alike. What transferred at 8B v4 was call-level, whether the action is something the user asked for, not a reading of the injected instruction. The untrained 8B agent recognises the same thing: it flags every InjecAgent attacker call as STOP (`unauthorized_scope` 1.000) and is at chance on injection.

The cause was a label leak in our generator; removing it is sufficient. Contrasting rendered calls between injected and non-injected synthetic examples showed that the v1–v5 generator wrote an `instructed_target` argument into every injected call and no other (presence 1.00 vs. 0.00 over 13,312 v4 rows) and drew the target from five literals, three containing “attacker” (Appendix H). The training signal was carried by an argument key; the observation never had to be read. v6 removes the key, fills the target argument for benign calls too, and renders 300–2,000 character multi-format observations with the instruction at a random position. Retrained on v6 with identical size and recipe, 2B reaches 0.693 AUPRC (AUROC 0.928) on AgentDojo injection and 8B 0.838 (0.966), and both heads fire (2B v6 median raw logit $+4.5$ for positives vs. -3.0). On InjecAgent both rank the poisoned variant above its clean twin on 97.7% of pairs, reaching 0.878 and 0.943 AUPRC on a label whose only evidence is in the untrusted text; stripping identity from 8B v6 leaves injection and pair win unchanged (0.953, 0.978), so the reading is content-driven. Meanwhile the synthetic injection AUPRC did not change (0.802 to 0.788 for 2B, 0.791 to 0.795 for 8B). The leak had not inflated the synthetic score; it had determined what the head learned to read, and no synthetic split could have shown it.

What does not transfer. `unauthorized_scope` is at base rate on AgentDojo for every checker (0.21–0.36 vs. 0.253); the label counts harmless auxiliary reads as out of scope. On InjecAgent, 8B v4 has signal (0.849 vs. 0.752 base) and 8B v6 does not (0.677) unless the port’s placeholder principal and scope fields are removed (0.917): the v6 head is misled by identity fields the port filled generically. Calibration does not transfer: macro ECE is 0.20–0.27 on AgentDojo against under 0.03 on synthetic data. Refitting isotonic calibrators on half of the AgentDojo calls brings the other half to macro ECE 0.008 (2B) and 0.010 (8B) with AUPRC unchanged, most of the gain by 250–500 labelled calls. A deployment needs a few hundred in-domain labels for its probabilities, not for its ranking. Over all four AgentDojo suites (30,143 calls), the 8B v4 injection transfer holds on banking, workspace, and slack (AUPRC 0.70, 0.75, 0.43 at base rates 0.15, 0.08, 0.19) and is weak on travel (0.13 at 0.025); 2B v4 is at base rate everywhere, and policy labels outside banking are one generic policy per suite and too weak to claim transfer on (Table 16).

9 RELATED WORK

LLM guardrails. Llama Guard (Inan et al., 2023), Granite Guardian (Padhi et al., 2024), Shield-Gemma (Zeng et al., 2024), NVIDIA Aegis (Ghosh et al., 2024), and gpt-oss-safeguard (OpenAI, 2025) classify content/harm over a conversation turn or action; Agent Action Guard (Pro-GenAI, 2025) is an embedding MLP over tool name and arguments with no identity or policy input. We do not claim they fail at their intended purpose; §3 shows they are at chance specifically on delegated-authority dimensions they were not trained to answer.

Agent safety benchmarks. AgentHarm (Andriushchenko et al., 2024), ToolEmu (Ruan et al., 2023), InjecAgent (Zhan et al., 2024), and AgentDojo (Debenedetti et al., 2024) measure end-to-end agent behavior or injection success with an outcome judge. They answer “did the agent do something bad,” not “should this action have been allowed given who it was for,” and emit no per-action calibrated probability a policy engine can compose. We reuse two as external test sets by deriving per-action labels from their own ground-truth functions.

Authorization engines and declared intent. OPA (Open Policy Agent), Cedar (Cedar Policy), and XACML (OASIS, 2013) decide allow/deny symbolically from a fully specified request and policy. OWASP’s agentic guidance (OWASP GenAI Security Project, 2025) and positions built on it (Capiscio, 2026) hold that authorization must be deterministic and auditable: the agent declares a typed mandate and an enforcement point checks each call against it. Our results support the premise (the agent is at chance on its own scope) and show the limit: once the mandate is natural-language policy, the symbolic checker (our rule baseline) drops to 0.33–0.50 AUPRC on `policy_conflict` and `unauthorized_scope`, and only a learned reader recovers it across paraphrases. forecheck keeps the decision deterministic and puts the learned component where that guidance allows it: the

scorer never authorises, it prices the review. Temperature scaling (Platt, 1999; Guo et al., 2017) and the Wilson interval (Wilson, 1927) are standard; we apply them to policy *composition*, where §5’s k -invariance is a function of calibration sharpness.

10 LIMITATIONS

Synthetic data. Every number outside §8 is on data from forecheck’s own generator and measures whether an arm learned the generator’s label function. No human-labelled or production-traffic evaluation exists and no real-world safety claim is made. The blind judge checks whether a label is observable in the text, not whether it is right; it finds `insufficient_context` unrecoverable, and every number on that dimension carries that caveat. The judge is one frontier model with one prompt.

Generator regularities. The model may exploit template regularities in how the generator writes clauses and entitlement lists. The phrasing and kind splits only partly rule this out: phrasing generalizes, kind does not, which is consistent with a lookup over kinds. The `privilege_escalation` fields were not rendered before v4, so every v2/v3 number on that dimension (0.46–0.53 AUPRC) reflects a data defect. The v1–v5 injection leak means every synthetic injection number before v6, and the judge’s $\kappa=0.905$ on it, is on text where a key-presence detector would score 1.0; the trained arms scored about 0.80, so the leak did not inflate the score but determined what was learned, and only the external pairs could reveal it. Retrained on v6, every other synthetic dimension is within 0.01 of v4 except `policy_conflict` on `heldout_policy_kind` (0.692 at 2B, inside the seed band; 0.767 at 8B, 0.08 below the v4 mean).

Seeds. Three 2B seeds on v4 and on v5 agree on macro AUPRC to within 0.001, but the `heldout_policy_kind``policy_conflict` cell spans 0.06 (sd 0.03), so smaller differences on that cell are noise. 8B has two seeds on v4 (test macro 0.956 and 0.948, the gap entirely on `insufficient_context`); both v6 checkers are single-seed, so 2B-vs-8B comparisons on v6 are unqualified.

External labels are partly self-written and uneven. AgentDojo’s policy predicates and InjecAgent’s five policies are ours; injection and scope labels are the benchmarks’ own. Banking has four tool-specific policies, the other suites one generic policy each, so policy transfer rests on banking and InjecAgent. AgentDojo’s `unauthorized_scope` counts harmless auxiliary reads as out of scope. The InjecAgent port fills principal and scope with placeholders, which the 8B v6 head reads as authorising the attacker call. The v6 checkers have not been run on the four-suite export. Calibration does not transfer without a few hundred in-domain labels.

Other. A single English template generator produces every synthetic example. The cost table in §5 is the default severity tier. Generator seed nondeterminism was found and fixed mid-project; v2/v3 data are not byte-reproducible from their configs.

11 CONCLUSION

Pre-execution risk for agent tool calls is largely a delegated-authority problem, not a content-safety problem, and general-purpose safety models do not solve it: removing identity and policy context from a trained checker collapses exactly the dimensions that depend on it, a blind judge loses the same dimensions, and zero-shot safety models and the acting agent are at chance on them with or without the context. Two properties decide whether a guardrail can detect authorization violations and survive deployment alongside several policies: delegated-authority context at its input and calibrated probabilities at its output. Policy-conditioned reasoning generalizes to new wording but not yet to new policy kinds; that gap, and a hand-labelled real-traffic probe, are what this line of work needs next. On the two external benchmarks the policy-conflict ranking held zero-shot, the probabilities needed a few hundred in-domain labels, and the injection result was wrong for a reason no synthetic split could show. The case for evaluating on data one did not generate is the strongest result here.

REFERENCES

- Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Eric Winsor, Eliot Wynne, Yarin Gal, and Xander Davies. Agentharm: A benchmark for measuring harmfulness of llm agents. *arXiv preprint arXiv:2410.09024*, 2024.
- CapiscIO. Automated intent classification for AI agents is an unsolved problem. <https://capisc.io/blog/automated-intent-classification-for-ai-agents-is-an-unsolved-problem>, 2026. Vendor position piece. Accessed 2026-09-22.
- Cedar Policy. Cedar policy language documentation. <https://www.cedarpolicy.com/en/documentation>. Accessed 2026-09-22.
- Edoardo DeBenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *arXiv preprint arXiv:2406.13352*, 2024.
- Shaona Ghosh, Prasoon Varshney, Erick Galinkin, and Christopher Parisien. Aegis: Online adaptive ai content safety moderation with ensemble of llm experts. *arXiv preprint arXiv:2404.05993*, 2024.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- IBM Granite Team. Granite 3.0 / 3.3 language models: Model card and technical report. Technical report, IBM Research, 2024. See also Granite 3.3 (2025) release notes.
- Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabza. Llama guard: Llm-based input-output safeguard for human-ai conversations. *arXiv preprint arXiv:2312.06674*, 2023.
- OASIS. extensible access control markup language (xacml) version 3.0. OASIS Standard, <http://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html>, 2013.
- Open Policy Agent. Open policy agent (opa) documentation. <https://www.openpolicyagent.org/docs/latest/>. Accessed 2026-09-22.
- OpenAI. gpt-oss-safeguard: Open-weight reasoning models for safety policy classification. Technical report, OpenAI, 2025. Model and technical report.
- OWASP GenAI Security Project. OWASP securing agentic applications guide. <https://genai.owasp.org/>, 2025. Accessed 2026-09-22.
- Inkit Padhi, Manish Nagireddy, Giandomenico Cornacchia, Subhajit Chaudhury, Tejaswini Pedapati, Pierre Dognin, Keerthiram Murugesan, Erik Miehl, Martin Santillan Qian, Ramya Ramamurthy, Giulio Zizzo, Ambrish Eltokhy, Michael Hind, Prasanna Ram, Payel Das, Pin-Yu Chen, Jerret Cunningham, Ananya Agarwal, Prasanna Sattigeri, Kush Raghavan, and Kush R. Varshney. Granite guardian. *arXiv preprint arXiv:2412.07724*, 2024.
- John Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, pp. 61–74, 1999.
- Pro-GenAI. Agent action guard: Classifying AI agent actions to ensure safety and reliability. Preprint, <https://doi.org/10.20944/preprints202510.1415.v2>; code <https://github.com/Pro-GenAI/Agent-Action-Guard>, 2025. Accessed 2026-09-22.

Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of lm agents with an lm-emulated sandbox. *arXiv preprint arXiv:2309.15817*, 2023.

Benjamin Warner, Antoine Chaffin, Benoît Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Nathan Cooper, Griffin Adams, Jeremy Howard, and Iacopo Poli. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *arXiv preprint arXiv:2412.13663*, 2024.

Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

Wenjun Zeng, Yuchen Liu, Ryan Mullins, Ludovic Peran, Joe Fernandez, Hamza Harkous, Karthik Narasimhan, Drew Proud, Piyush Kumar, Bhaktipriya Radharapu, Olivia Sturman, and Oscar Wahltinez. Shieldgemma: Generative ai content moderation based on gemma. *arXiv preprint arXiv:2407.21772*, 2024.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. *arXiv preprint arXiv:2403.02691*, 2024.

A IDENTITY ABLATION: ALL DIMENSIONS, v2

Table 8 gives the v2 per-dimension numbers behind Figure 1. The v4 ablation (Table 2) replicates the pattern with one change: `privilege_escalation` survives stripping in v4 (1.000 to 1.000) because the v4 renderer puts the `deciding_authority_before/authority_after` fields into the tool-call text; in v2 they were never rendered and every arm scored 0.46–0.53 on it, so it is omitted below.

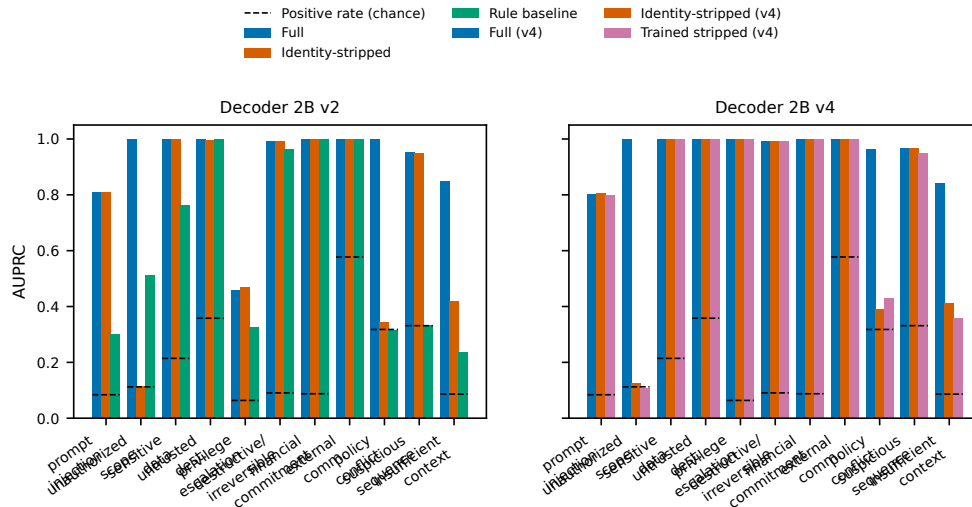


Figure 1: Decoder 2B v2, per-dimension AUPRC on heldout_family ($n=6640$): full model vs. identity-stripped vs. rule baseline, with each dimension’s positive rate (chance) as a dashed line. The drop is confined to `unauthorized_scope`, `policy_conflict`, and `insufficient_context`.

Table 8: Decoder 2B v2, heldout_family ($n=6640$) AUPRC per dimension: full, identity-stripped, and rule baseline. Source: docs/results/synthetic-v2/README.md.

Dimension	Decoder (full)	Decoder (stripped)	Rule baseline
unauthorized_scope	1.000	0.116	0.512
policy_conflict	0.999	0.345	0.315
insufficient_context	0.849	0.418	0.235
prompt_injection_influence	0.810	0.810	0.301
suspicious_action_sequence	0.952	0.950	0.334
financial_commitment	1.000	1.000	1.000

Table 9: Decoder 2B, macro AUPRC and policy_conflict (pc) AUPRC across splits for the v3 (15 policy kinds, 2 withheld), v4 (same data, privilege_escalation render fix), and v5 (30 kinds, 4 withheld) generators, seed 0; last column an 8B LoRA on v4 data, seed 0. Three-seed values for the heldout_policy_kind pc cell: v4 0.752, 0.689, 0.711; v5 0.704, 0.755, 0.717; 8B v4 (two seeds) 0.851, 0.825. Source: docs/results/synthetic-v2/README.md.

Split	v3 macro	v4 macro	v5 macro	v3 pc	v4 pc	v5 pc	8B v4 pc
test	0.908	0.960	0.957	0.994	0.994	0.996	0.990
heldout_family	0.914	0.960	0.951	0.941	0.964	0.933	0.965
heldout_policy_kind	0.889	0.944	0.941	0.653	0.752	0.704	0.851
heldout_policy_phrasing	0.921	0.968	0.970	0.987	0.993	0.980	0.978

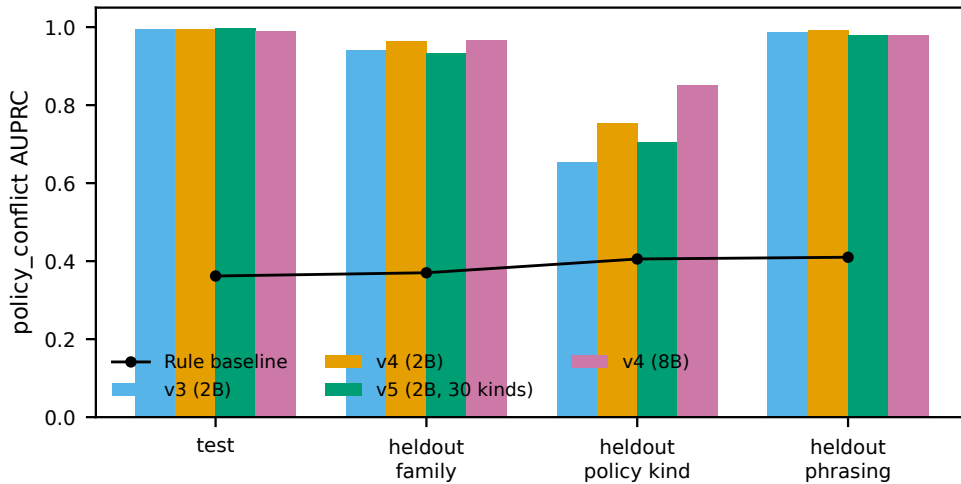


Figure 2: Decoder 2B, policy_conflict AUPRC across splits for the v3, v4, and v5 generators, plus the 8B LoRA on v4 data, with the rule baseline's policy_conflict AUPRC on each split shown as a line. Bars for test, heldout_family, and heldout_policy_phrasing sit near the ceiling; heldout_policy_kind drops toward, though still above, the rule baseline, for every generator and both sizes.

B POLICY GENERALIZATION: MACRO AUPRC AND PER-SEED VALUES

The v4 render fix also confirms that `privilege_escalation`'s earlier AUROC-high/AUPRC-low signature was a data defect, not a model limit: rendering the deciding fields lifts it from 0.505 to 1.000 AUPRC on `heldout_family`.

C MULTI-POLICY STACKING: FULL TABLE AND CURVES

Table 10 extends Table 4 with the `joint` strategy (rules from all k policies merged into one bundle and evaluated once in threshold mode), false negative rate, and review rate. `independent` and `joint` are identical at $k \leq 2$ because the synthetic stacking policies use only plain rules, so the engine's combination logic has nothing to do differently; they diverge at $k=3$ only through threshold interaction. The $k=1$ row already trades false positives for false negatives relative to the F1-optimal threshold (FPR/FNR 0.058/0.170 vs. 0.015/0.253) because the default cost table is severity-weighted.

Table 10: Multi-policy stacking, `heldout_family` ($n=6640$). Source: docs/evaluation/multi-policy-stacking.md.

Arm	k	Strategy	FPR	FNR	Review rate
Decoder 2B	1	any	0.015	0.253	0.31
Decoder 2B	2	independent	0.140	0.080	0.30
Decoder 2B	2	expected_cost_joint	0.082	0.070	0.28
Decoder 2B	3	independent	0.966	0.016	0.66
Decoder 2B	3	joint	0.905	0.051	0.63
Decoder 2B	3	expected_cost_joint	0.082	0.070	0.28
Decoder 2B	8	independent	0.966	0.007	0.58
Decoder 2B	8	expected_cost_joint	0.082	0.070	0.28
Encoder granite-r2	2	independent	0.710	0.042	0.26
Encoder granite-r2	2	expected_cost_joint	0.921	0.010	0.37
Encoder granite-r2	3	independent	0.990	0.008	0.21
Encoder granite-r2	3	expected_cost_joint	0.921	0.010	0.37

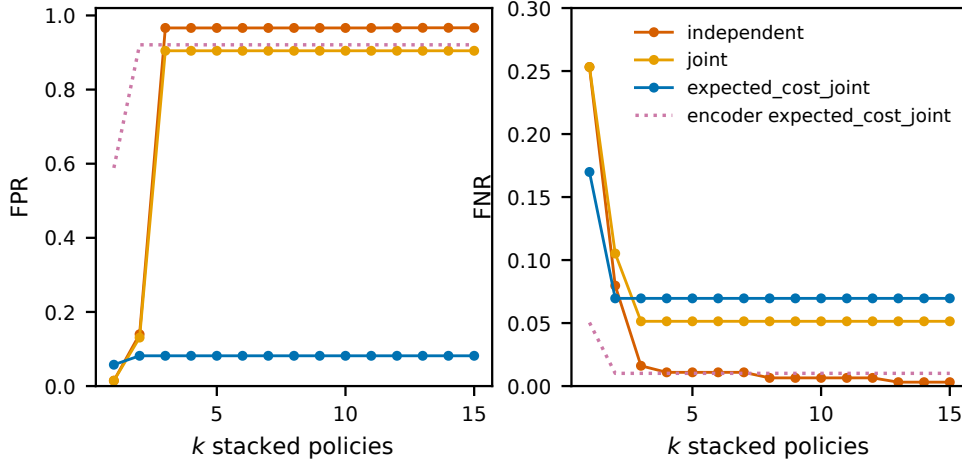


Figure 3: Decoder 2B, FPR (left) and FNR (right) vs. number of stacked policies k on `heldout_family` ($n=6640$), for independent, joint, and expected_cost_joint strategies; the granite-embedding-r2 encoder's expected_cost_joint curve is dotted. The flat decoder curve is a property of its calibration, not of the decision rule: the encoder's curve under the same rule sits near the top of the FPR panel.

D APPROVAL ELIMINATION

The product-facing metric is how many of a deployment’s human-approval reviews can be automated at a chosen incident rate: sort examples by the decoder’s expected cost of ALLOW and ask what fraction of the review queue can be auto-decided while the fraction of auto-allowed actions that are actually risky stays under a budget. Because `heldout_family` is adversarial-heavy (58% base incident rate), Table 11 reports both the raw curve and a curve importance-reweighted to a 5% deployment incident rate, with elimination computed conservatively via a 95% Wilson upper bound (Wilson, 1927) on the realized incident rate in the auto-allowed set.

Table 11: Decoder 2B, approval elimination on `heldout_family` (v2 $n=6640$, v4 $n=6653$), bundle balanced. “raw” is the split’s own 57–58% base incident rate; “rw” reweights rows so the incident rate is 5%. Source: `docs/evaluation/approval-elimination.md`.

Budget	Elim. (v2, raw)	Elim. (v2, rw)	Elim. (v4, rw)	Review (v4, rw)
0.1%	0.000	0.000	0.000	0.969
0.5%	0.000	0.000	0.000	0.969
1%	0.000	0.888	0.903	0.067
2%	0.000	0.961	0.952	0.021
5%	0.000	0.992	0.992	0.001

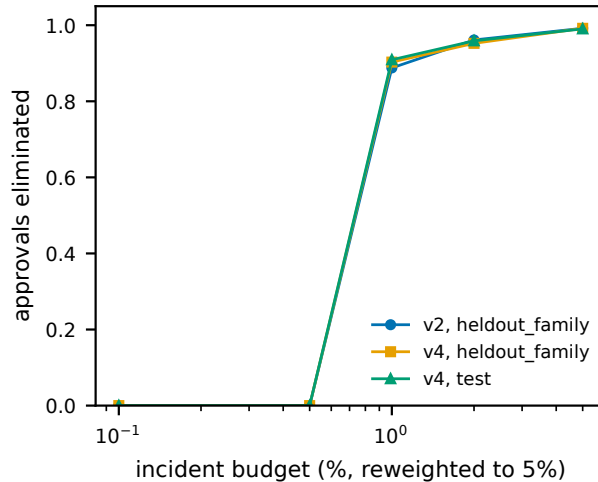


Figure 4: Approvals eliminated vs. incident budget, reweighted to a 5% deployment incident rate, decoder 2B v2 and v4 on `heldout_family` and v4 on test, log-scaled budget axis. The three curves nearly overlap.

At a 5% deployment incident rate the calibrated decoder auto-allows 88.8–99.2% of actions that would otherwise need review while keeping the allowed-but-risky rate under budget. The unweighted column is near zero because the split’s top-ranked rows already contain risky examples by construction; it is a ceiling of the split’s design, not of the classifier. Below a 1% budget the Wilson bound cannot be met at this sample size and the conservative curve reports zero.

E AGENT SELF-JUDGMENT: ALL DIMENSIONS

The checker beats the agent’s single-verdict score on all eleven dimensions by 0.45–0.90 AUPRC. The small identity signal that does exist in the agent (`unauthorized_scope` AUROC 0.61 full vs. 0.48 stripped) shows it reads the context a little without turning it into a usable decision. The 2B agent-self condition behaves the same way and is omitted.

702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755

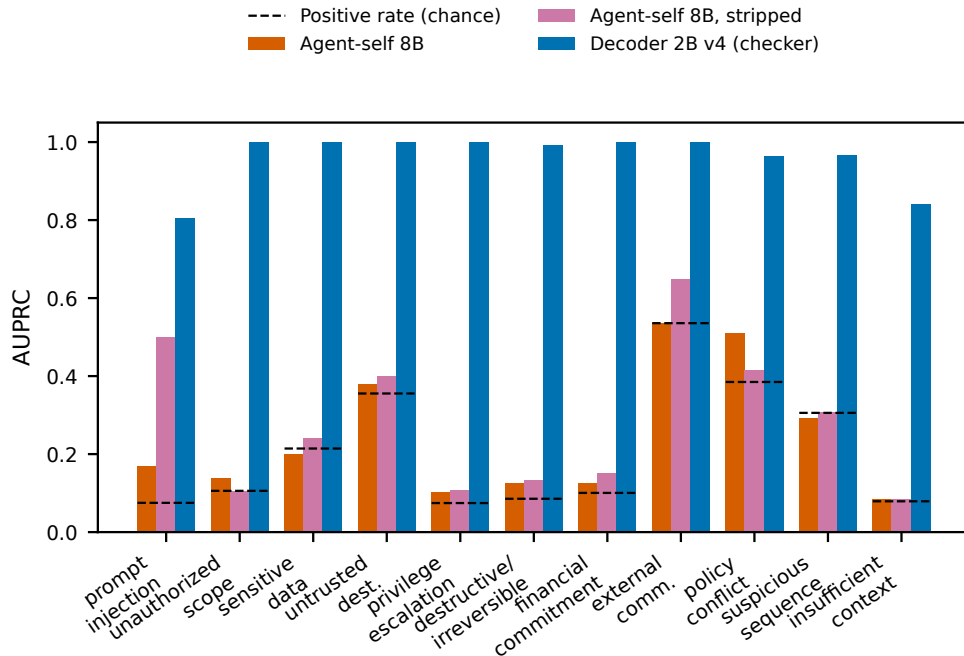


Figure 5: Per-dimension AUPRC on heldout_family ($n=2000$ for the agent-self-8B rows, $n=6653$ for the decoder), positive rate as a dashed line. The agent’s bars track chance everywhere except policy_conflict and external_communication; the external checker clears 0.84 on every dimension.

F BLIND JUDGE: FULL AGREEMENT TABLE

Table 12: Independent LLM-judge agreement with the generator label, yes vs. not-yes, $n=600$ v4 rows (gpt-5.6-sol, blind to the generator’s latent scenario), full and identity-stripped renderings. Agreement/recall/precision are for the full-context pass. Source: docs/results/judge/README.md.

Dimension	κ full	κ stripped	Agreement	Judge recall	Judge precision
privilege_escalation	0.909	0.909	0.988	1.00	0.84
prompt_injection_influence	0.905	0.905	0.983	0.84	1.00
policy_conflict	0.714	0.029	0.862	0.89	0.78
destructive_or_irreversible_action	0.511	0.542	0.897	0.60	0.54
suspicious_action_sequence	0.404	0.428	0.852	0.82	0.33
unauthorized_scope	0.391	0.040	0.800	0.74	0.38
financial_commitment	0.359	0.300	0.935	0.24	0.92
external_communication	0.331	0.361	0.897	0.86	0.23
untrusted_destination	0.213	0.205	0.777	0.41	0.29
insufficient_context	0.059	-0.036	0.843	0.14	0.15
sensitive_data_exposure	0.002	0.061	0.783	0.17	0.08

G EXTERNAL BENCHMARKS: FULL TABLES

Per-example logits. The 2B v4 injection head never fires on AgentDojo: raw logits for all 4,133 evaluable calls lie between -5.0 and -3.0 , the highest-scoring positive is -3.25 , and no call scores above zero, while on the same rows its policy_conflict head has median raw logit $+2.5$ for

756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809

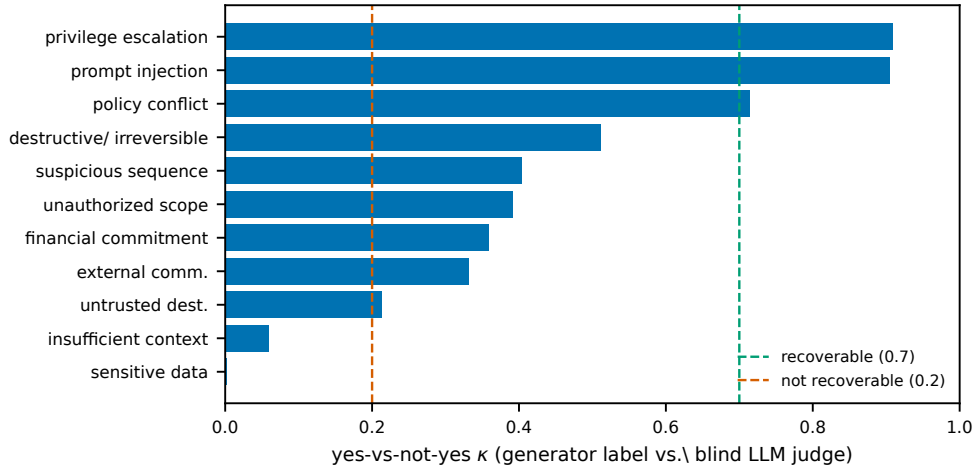


Figure 6: Yes-vs-not-yes κ per dimension, sorted, with the recoverable (0.7) and not-recoverable (0.2) tier lines.

Table 13: AUPRC against the judge’s labels on the judged heldout_family rows ($n=257$), decoder 2B v4 vs. rule baseline, with the decoder’s AUPRC against the generator’s own labels on the full split for reference. The lookup dimensions collapse for both arms because the judge defines them differently (e.g. financial_commitment requires funds to move; sensitive_data_exposure requires a lower-sensitivity destination), so they are not evidence for or against either arm.

Dimension	Decoder 2B v4	Rule baseline	Decoder vs. generator labels
prompt_injection_influence	1.000	0.459	0.804
privilege_escalation	0.977	0.297	1.000
policy_conflict	0.889	0.502	0.964
untrusted_destination	0.864	0.785	1.000
destructive_or_irreversible_action	0.635	0.509	0.992
unauthorized_scope	0.592	0.509	0.999
suspicious_action_sequence	0.528	0.467	0.966
external_communication	0.437	0.381	1.000
sensitive_data_exposure	0.405	0.291	1.000
financial_commitment	0.327	0.280	1.000
insufficient_context	0.090	0.090	0.841
macro	0.613	0.415	0.960

Table 14: AUPRC on AgentDojo banking traces, zero-shot, $n=4,215$ ($n=4,133$ for injection). `untrusted_destination` has positives only and is excluded from macro. Guardian and agent-self rows are on a fixed 1,000-row subsample. Macro ECE of the calibrated arms is 0.20–0.27 here against under 0.03 on synthetic data. Source: `docs/results/agentdojo/README.md`.

Arm	injection	scope	policy	financial	destructive	macro
Rule baseline	0.250	0.253	0.237	0.896	1.000	0.527
Decoder 2B v4	0.128	0.257	0.849	1.000	1.000	0.647
2B v4, eval-stripped	0.125	0.218	0.694	1.000	1.000	0.608
2B v4, train-stripped	0.150	0.234	0.390	1.000	1.000	0.555
Decoder 8B v4	0.701	0.355	0.626	0.923	0.999	0.721
8B v4, eval-stripped	0.743	0.266	0.404	0.934	1.000	0.669
Decoder 2B v6 (leak-free)	0.693	0.214	0.670	0.935	1.000	0.702
2B v6, eval-stripped	0.669	0.306	0.521	0.941	1.000	0.687
Decoder 8B v6 (leak-free)	0.838	0.239	0.785	0.945	0.831	0.728
8B v6, eval-stripped	0.819	0.354	0.450	0.979	0.975	0.715
Granite Guardian 3.3 8B	0.103	0.289	0.224	0.257	0.217	0.218
Agent-self 8B (ALLOW/STOP)	0.291	0.442	0.449	0.591	0.105	0.375

Table 15: AUPRC on InjecAgent, zero-shot, $n=4,250$ ($n=3,196$ for injection, positive rate 0.500). Pair win: fraction of the 1,598 poisoned/clean pairs on which the poisoned variant scores higher on injection; tie: fraction with identical raw score. Baseline rows on a fixed 1,000-row subsample (injection $n=767$). Source: `docs/results/injecagent/README.md`.

Arm	injection	scope	policy	financial	destructive	macro	pair win	tie
Rule baseline	0.500	0.752	0.464	0.683	1.000	0.680	–	–
Decoder 2B v4	0.549	0.732	0.965	1.000	1.000	0.849	0.504	0.239
Decoder 8B v4	0.544	0.849	0.977	0.734	1.000	0.821	0.353	0.385
8B v4, eval-stripped	0.556	0.843	0.968	0.766	1.000	0.827	0.382	0.397
Decoder 2B v6 (leak-free)	0.878	0.704	0.922	0.970	0.998	0.894	0.977	0.005
Decoder 8B v6 (leak-free)	0.943	0.677	0.965	0.771	0.997	0.870	0.977	0.012
8B v6, eval-stripped	0.953	0.917	0.963	0.839	0.998	0.934	0.978	0.016
Granite Guardian 3.3 8B	0.511	0.613	0.359	0.102	0.197	0.356	–	–
Agent-self 8B (ALLOW/STOP)	0.530	1.000	0.647	0.061	0.178	0.483	–	–

Table 16: AgentDojo, all four suites (30,143 proposed calls: banking 4,215, slack 8,246, travel 10,701, workspace 6,981), `prompt_injection_influence` AUPRC / AUROC per suite, positive rate in the header. v4 checkers only; the v6 checkers have not been run on this export. Source: `docs/results/agentdojo/README.md`.

Arm	banking (0.150)	slack (0.186)	travel (0.025)	workspace (0.077)
Rule baseline	0.250 / 0.737	0.222 / 0.601	0.021 / 0.360	0.138 / 0.740
Decoder 2B v4	0.128 / 0.348	0.178 / 0.479	0.048 / 0.686	0.065 / 0.353
Decoder 8B v4	0.701 / 0.895	0.433 / 0.738	0.131 / 0.834	0.752 / 0.941
8B v4, eval-stripped	0.743 / 0.915	0.421 / 0.726	0.146 / 0.844	0.727 / 0.943
Agent-self 8B	0.354 / 0.832	0.292 / 0.687	0.026 / 0.484	0.246 / 0.866

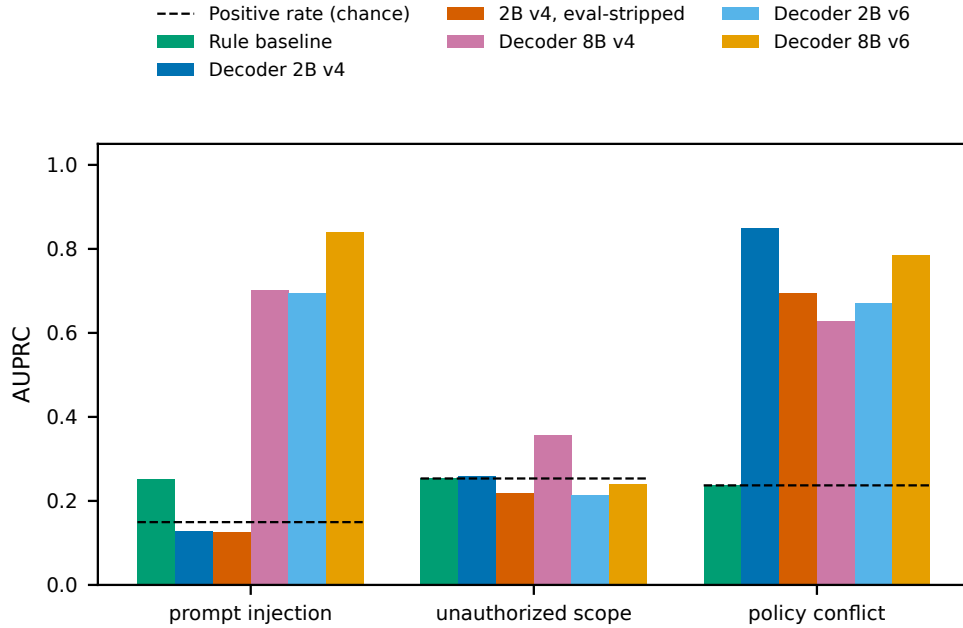


Figure 7: AUPRC on AgentDojo banking traces for the three learned dimensions: rule baseline, decoder 2B v4, 2B v4 eval-stripped, decoder 8B v4, and the leak-free 2B v6 and 8B v6, zero-shot, with each dimension’s positive rate as a dashed line.

positives and -3.5 for negatives. The 8B v4 head is live but shifted (median -1.75 for positives vs. -3.50 for negatives, 29% of positives above zero vs. 0.7% of negatives). After the v6 retrain, 2B v6 has median $+4.5$ vs. -3.0 (92% vs. 23% above zero) and 8B v6 $+6.25$ vs. -3.25 (94% vs. 8%). On InjecAgent the v4 heads sit at raw logit -4 on poisoned and clean alike; the v6 heads have median raw $+3.5$ vs. -6.25 (2B) and $+2.25$ vs. -3.5 (8B), and 58% of poisoned calls score above 0 at both sizes against 9% (2B) and 0.1% (8B) of clean. Inference is deterministic (per-example raw difference 0 between repeated runs).

Recalibration. Refitting per-dimension isotonic calibrators on a random half of the AgentDojo banking calls and testing on the other half (five seeds) gives macro ECE 0.008 (2B) and 0.010 (8B) with AUPRC unchanged; the 8B `unauthorized_scope` head goes from ECE 0.60 to 0.01 by learning to output the base rate. Most of the gain is reached with 250–500 labelled calls (`scripts/agentdojo_recalibrate.py`, `docs/results/agentdojo_recalibration.md`).

Other suites. `unauthorized_scope` outside banking is carried by the rule baseline (0.749 on slack, 0.733 on workspace, where nearly half of all calls are labelled out of scope); 2B adds under 0.09 and both 8B arms are below the rule. `policy_conflict` on slack, travel, and workspace uses one generic policy per suite at 0.7–8% base rates: 2B AUROC is 0.78–0.89 but AUPRC 0.005–0.49, so we claim policy transfer on banking and InjecAgent only.

H INJECTION LABEL LEAK: DIAGNOSIS AND FIX

After the 2B checker failed on AgentDojo injection we contrasted the rendered call between injected and non-injected synthetic examples. The v1–v5 generator wrote an `instructed_target` argument into every injected call and into no other call (presence 1.00 vs. 0.00 over 13,312 v4 rows), put a `to` argument into 100% of injected calls vs. 3% of others, and drew the injected target from five literals, three containing “attacker” (`attacker-agent-007`, `external-support@attacker.example`, `shadow-ops@attacker.example`). Injected observations were five short templates (median 132 characters) against AgentDojo’s 2,000-

918 character record dumps. The v6 renderer (2.0.0) removes `instructed_target`, chooses the tar-
919 get argument key from the tool kind and fills it for benign calls too, draws targets from pools shared
920 between injected and benign calls, and renders 300–2,000 character multi-format observations with
921 the instruction at a random position. A leakage check on argument-key presence parity and a
922 token denylist runs before training. Judge agreement on `prompt_injection_influence`
923 ($\kappa=0.905$, Table 12) was measured on v4 text and has not been re-run on v6. Full write-up:
924 `docs/results/notes/injection-label-leak-diagnosis.md`.

925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971